

**ACTIVIDAD DE APRENDIZAJE UNIDAD 3: SISTEMA DE RESTAURANTE
CON COLECCIONES JAVA, MAP Y STREAM**

ESTUDIANTE:

REY DAVID BORREGO PEREZ

ASIGNATURA:

ESTRUCTURA DE DATOS

DOCENTE:

JHON ARRIETA ARRIETA

PROGRAMA:

INGENIERÍA DE SOFTWARE

UNIVERSIDAD DE CARTAGENA

2026-1

1. Introducción

La Unidad 3 profundiza en el uso de estructuras provistas por el SDK de Java para resolver problemas de gestión. En este informe se desarrolla el ejercicio asignado número 4, correspondiente al caso Restaurante, con entidad principal Pedido e identificador número de pedido. A diferencia de la Unidad 2, aquí no se implementan nodos, listas, colas ni pilas propias; se utilizan colecciones estándar como List, Queue, Deque, Map y operaciones funcionales con Stream.

El sistema mantiene el mismo contexto funcional: los pedidos se registran, quedan pendientes, se procesan y se almacenan en un historial. Sin embargo, la implementación mejora la productividad y reduce errores al apoyarse en estructuras probadas de Java. Además, Stream permite expresar búsquedas, filtros, ordenamientos, conteos y agrupamientos de forma clara y declarativa (Bloch, 2018; Oracle, n.d.).

Este documento sigue una estructura académica tipo APA, similar al informe de la Unidad 1: introducción, objetivos, justificación, desarrollo, repositorio, video, conclusiones y referencias. También incluye la comparación conceptual obligatoria entre estructuras personalizadas y colecciones del SDK.

2. Objetivos

Objetivo General

Desarrollar un sistema de gestión de pedidos para restaurante en Java, utilizando List, Queue, Deque, Map y Stream, para administrar el registro, atención, historial, búsqueda, filtrado, ordenamiento, agrupamiento y estadísticas de pedidos.

Objetivos Específicos

- Modelar la clase Pedido con cinco atributos, constructor, getters, setters, toString(), equals() y hashCode().
- Usar List como registro general de todos los pedidos ingresados al sistema.
- Usar Queue para administrar los pedidos pendientes bajo el principio FIFO.
- Usar Deque como pila moderna para el historial de pedidos procesados.
- Usar Map para búsqueda rápida por número de pedido y validación de duplicados.
- Aplicar Stream para buscar, filtrar, ordenar, transformar, contar y agrupar pedidos.
- Comparar el uso de colecciones estándar con las estructuras personalizadas de la unidad anterior.

3. Justificación

Las colecciones del SDK de Java son herramientas esenciales en el desarrollo profesional porque permiten resolver problemas comunes de almacenamiento y consulta sin implementar estructuras desde cero. Esto mejora la mantenibilidad, reduce el riesgo de errores y permite concentrarse en la lógica del negocio. En el caso del restaurante, las colecciones permiten separar claramente el registro general, la cola de pendientes, el historial y el índice de búsqueda rápida.

La incorporación de Stream es importante porque facilita operaciones de consulta y reporte sobre colecciones. En lugar de escribir ciclos repetitivos para cada filtro o conteo, se pueden construir expresiones declarativas que indican qué se quiere obtener: pedidos por estado, clientes ordenados, agrupamientos por categoría o conteos por estado. Este enfoque mejora la legibilidad cuando se usa de manera moderada y con propósito.

4. Desarrollo

4.1 Logros de aprendizaje

Durante la actividad se aprendió a diferenciar entre implementar estructuras propias y utilizar estructuras provistas por Java. En la Unidad 2, el control interno era mayor porque se manipulaban nodos y referencias; en la Unidad 3, la productividad aumenta porque Java ofrece implementaciones estables y optimizadas.

Se comprendió que List es útil para conservar todos los elementos, Queue para gestionar pendientes en orden FIFO, Deque para actuar como pila moderna bajo LIFO y Map para acceder rápidamente a un pedido mediante su número. También se fortaleció el uso de Stream con operaciones como filter(), map(), sorted(), collect(), toList(), count(), findFirst(), anyMatch(), allMatch(), noneMatch(), groupingBy(), counting() y toMap().

Una dificultad relevante fue decidir cuándo usar Map y cuándo usar Stream. La regla aplicada fue usar Map para buscar por identificador principal, porque ofrece acceso rápido por clave; y usar Stream para búsquedas por otros criterios, filtros, ordenamientos, estadísticas y agrupamientos.

4.2 Descripción funcional del sistema

El sistema permite registrar pedidos de restaurante con número, cliente, descripción, categoría, total y estado. Al registrar un pedido, se agrega simultáneamente a la List general, a la Queue de pendientes y al Map de búsqueda rápida. El estado inicial siempre es PENDIENTE.

Cuando se procesa un pedido, se retira de la Queue mediante poll(), se actualiza su estado a PROCESADO y se guarda en el Deque mediante push(). Si se deshace el último procesamiento, se extrae del historial con pop(), se cambia nuevamente a PENDIENTE y se retorna a la Queue.

Tabla 1. Colecciones usadas en el sistema de restaurante.

Necesidad del sistema	Colección utilizada	Justificación
Registro completo	List<Pedido>	Permite conservar y recorrer todos los pedidos.
Pendientes	Queue<Pedido>	Respetar el orden FIFO de atención.
Historial	Deque<Pedido>	Funciona como pila LIFO moderna.
Búsqueda rápida	Map<String, Pedido>	Permite consultar por número de pedido.
Reportes	Stream	Permite filtrar, ordenar, contar y agrupar.

4.3 Componentes desarrollados

Tabla 2. Clases desarrolladas para la Unidad 3.

Clase	Responsabilidad
Pedido	Representa el pedido con número, cliente, descripción, categoría, total y estado.
GestorRestaurante	Administra las colecciones, validaciones, búsquedas, filtros y operaciones de negocio.
Main	Presenta el menú de consola, captura datos y llama los métodos del gestor.

4.4 Uso de List, Queue, Deque y Map

```
private final List<Pedido> pedidos;
private final Queue<Pedido> pendientes;
private final Deque<Pedido> historial;
private final Map<String, Pedido> indicePorNumero;
```

El registro de un pedido demuestra el uso combinado de las colecciones: se valida que el número no exista en el Map, se cambia el estado a PENDIENTE, se agrega a la List, se encola en la Queue y se indexa en el Map.

```
public void registrarPedido(Pedido pedido) {
    validarPedido(pedido);
    if (indicePorNumero.containsKey(pedido.getNumeroPedido())) {
        throw new IllegalArgumentException("Ya existe un pedido con ese numero.");
    }
    pedido.setEstado("PENDIENTE");
    pedidos.add(pedido);
    pendientes.offer(pedido);
    indicePorNumero.put(pedido.getNumeroPedido(), pedido);
}
```

4.5 Uso de Stream

Stream se aplica en operaciones de consulta y reporte. Por ejemplo, buscar por cliente usa filter() y findFirst(); filtrar por estado usa filter() y toList(); ordenar por total usa sorted(); obtener clientes usa map(); y las estadísticas usan collect() con Collectors.groupingBy() y Collectors.counting().

```
public Optional<Pedido> buscarPorClienteStream(String cliente) {
    return pedidos.stream()
        .filter(p -> p.getCliente().equalsIgnoreCase(cliente))
        .findFirst();
}

public Map<String, Long> contarPorEstado() {
    return pedidos.stream()
        .collect(Collectors.groupingBy(Pedido::getEstado, Collectors.counting()));
}
```

4.6 Comparación conceptual con la Unidad 2

Tabla 3. Comparación entre Unidad 2 y Unidad 3.

Aspecto	Estructuras personalizadas	Colecciones del SDK Java
Implementación	El estudiante implementa la estructura.	Java provee la estructura.
Nodos	Se manipulan manualmente.	No se manipulan directamente.

Métodos	Se crean métodos propios.	Se usan métodos estándar.
Riesgo de errores	Mayor por referencias manuales.	Menor por APIs probadas.
Control interno	Mayor, se observa la lógica interna.	Menor, se delega en el SDK.
Productividad	Menor.	Mayor.
Mantenibilidad	Depende del código del estudiante.	Mayor por uso de APIs estándar.
Propósito	Comprender el funcionamiento interno.	Resolver problemas reales con herramientas del lenguaje.

4.7 Flujo del sistema

Registro:

List.add(pedido) + Queue.offer(pedido) + Map.put(numero, pedido)

Procesamiento:

Queue.poll() → estado PROCESADO → Deque.push(pedido)

Deshacer:

Deque.pop() → estado PENDIENTE → Queue.offer(pedido)

Reportes:

List.stream() → filter/map/sorted/collect/count/groupingBy

4.8 Evidencia de ejecución

Evidencia de ejecución por consola

```

===== SISTEMA RESTAURANTE CON COLECCIONES JAVA =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por identificador usando Map
7. Buscar elemento por otro criterio usando Stream
8. Filtrar elementos usando Stream
9. Ordenar elementos usando Stream
10. Ver estadísticas usando Stream y Map
11. Ver agrupamientos usando Stream y Map
12. Cancelar elemento pendiente
13. Deshacer último procesamiento
14. Ver cantidad de elementos
15. Salir
Seleccione una opción: Pedido{numeroPedido='P001', cliente='Ana Torres', descripcion='Hamburguesa, papas y jug
Pedido{numeroPedido='P002', cliente='Luis Mora', descripcion='Pizza personal y gaseosa', categoria='Domicilio'
Pedido{numeroPedido='P003', cliente='Marta Rios', descripcion='Ensalada y limonada', categoria='Para llevar',

===== SISTEMA RESTAURANTE CON COLECCIONES JAVA =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por identificador usando Map
7. Buscar elemento por otro criterio usando Stream
8. Filtrar elementos usando Stream
9. Ordenar elementos usando Stream
10. Ver estadísticas usando Stream y Map
11. Ver agrupamientos usando Stream y Map
12. Cancelar elemento pendiente
13. Deshacer último procesamiento
14. Ver cantidad de elementos
15. Salir
Seleccione una opción: Pedido{numeroPedido='P001', cliente='Ana Torres', descripcion='Hamburguesa, papas y jug
Pedido{numeroPedido='P002', cliente='Luis Mora', descripcion='Pizza personal y gaseosa', categoria='Domicilio'
Pedido{numeroPedido='P003', cliente='Marta Rios', descripcion='Ensalada y limonada', categoria='Para llevar',
Total pendientes: 3
Siguiente en cola: Pedido{numeroPedido='P001', cliente='Ana Torres', descripcion='Hamburguesa, papas y jugo',

```

Figura 1. Ejecución del sistema con List, Queue, Deque, Map y Stream.

5. Repositorio Público de GitHub

El código fuente debe publicarse en un repositorio público de GitHub. Para cumplir la exigencia de evidencia de construcción, el repositorio debe mostrar commits progresivos, mensajes descriptivos y evolución real del desarrollo. En este espacio se debe pegar el enlace definitivo después de subir el proyecto:

https://github.com/ReyDavid07/Actividad_de_AprendizajeU3_ED_UDEC.git

6. Conclusiones

La Unidad 3 permitió comprobar que las colecciones estándar de Java facilitan la solución de problemas reales porque ofrecen estructuras confiables, expresivas y fáciles de mantener. En el sistema de restaurante, cada colección cumple una función concreta y se integra con las demás sin necesidad de manipular nodos manualmente.

El uso de Map resultó especialmente útil para búsquedas por número de pedido, mientras que Stream aportó claridad en consultas por cliente, filtros por estado o categoría, ordenamientos, transformaciones y estadísticas. Esta combinación permite que el sistema no solo almacene datos, sino que también genere información útil para la toma de decisiones.

La comparación con la Unidad 2 mostró que las estructuras propias son valiosas para aprender el funcionamiento interno, pero las colecciones del SDK son más convenientes en proyectos reales por productividad, mantenibilidad y menor probabilidad de error.

7. Referencias

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Java (6th ed.). Wiley.

Oracle. (n.d.). Java Platform, Standard Edition API specification.
<https://docs.oracle.com/en/java/javase/>

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.